

Grand Challenge Report

SIDs: 550688934, 550817486, 550155126, 550714901, 550773919, 510427243

I. INTRODUCTION (ADDRESSING AND IMPORTANT APPLICATION)

Flooding remains one of Australia's most persistent and damaging natural disasters, regularly disrupting lives and infrastructure, particularly along the eastern seaboard. In recent years, catastrophic events such as the 2022 Lismore floods have underscored the need for predictive technologies capable of providing early warnings. These events not only inflict economic damage but also pose serious risks to human safety and the environment. According to hydrological data from the CAMELS-AUS dataset, many catchments in Australia exhibit high flashiness indices and short response times, which make early detection systems especially important. This project addresses the urgent need for flood detection and prediction by developing a Python-based data science solution that processes hydrological and meteorological data to identify and predict flood events. By equipping civil engineers with computational tools that forecast flood risk, this work contributes to the broader goals of resilience engineering, infrastructure design, and disaster preparedness. We primarily focused on five catchments from the NICHOLSON-LEIDCHHARDT RIVERS and FLINDERS-NORMAN RIVERS.

This research specifically contributes to computational civil engineering by demonstrating how Python-based models can provide lightweight, interpretable flood forecasts using openly available meteorological data. While many large-scale hydrological models rely on complex numerical simulation and geospatial datasets, our approach balances accessibility and accuracy, suitable for use in low-resource planning settings or as a prototype for real-time alert systems. The methodology is extensible to other hydrological hazards including drought and flash runoff, highlighting its relevance to water-sensitive urban design (WSUD).

II. COMPUTATIONAL PROBLEM FORMULATION

A. What computational problems are to be solved

This study aims to solve a clear two-fold computational problem: (i) how to detect historical flood events using streamflow thresholding, and (ii) how to predict whether a flood will occur on the following day based on recent weather conditions. Our approach is grounded in the principle of empirical hydrology, leveraging the 95th percentile of historical streamflow for each catchment as a proxy flood threshold. This percentile-based method accommodates inter-catchment variability without requiring pre-defined flood heights or complex hydraulic models.

B. What datasets and catchments were used

The datasets used were drawn from the CAMELS-AUS v2 collection [1], which includes daily hydrometeorological variables for 561 Australian catchments. We selected five

representative catchments based on data completeness: 912101A, 912105A, 915011A, 915206A, and 917107A. From each, we extracted daily streamflow (ML/day), precipitation (mm/day), and maximum temperature ($^{\circ}\text{C}/\text{day}$) spanning 1950–2022. Additionally, we incorporated catchment metadata (e.g. response time, flashiness index, baseflow index) for model evaluation.

To formulate the prediction model, we engineered time series features:

Lagged rainfall (1 and 2 days): Captures rainfall accumulation dynamics

Lagged temperature: Assesses thermal contribution to evapotranspiration and runoff

Rainfall 3-day sum: Proxy for cumulative storm impact

These features are structured as a NumPy array input X , with binary labels y indicating flood status on the following day. The supervised learning task was implemented as a binary classification problem using logistic regression. In this case, the model answers a simplified but practical question: Is a flood likely to occur tomorrow? This binary decision is often more actionable for disaster response units than probabilistic forecasts, particularly when deployed in community alerting systems. So, ultimately the model would communicate whether a flood is expected. A simple "Yes" or "No" is often the most useful output, as individuals in flood-prone areas primarily want to know if they need to prepare for a flood tomorrow. They are typically less concerned with the exact probability, what matters most is having a clear, actionable answer.

III. PROGRAMMING: BLOCK DIAGRAM, CODE STRUCTURE, AND KEY COMPONENTS

The programming workflow is structured into a modular pipeline, integrating data preprocessing, feature extraction, model training, evaluation, and prediction. And the data structure used is suitable due to its mutability and two-dimensional format. Its flexibility allows for efficient modifications, such as adding new columns (e.g., a "Flood" or "Predicted" column) and removing unwanted entries (such as placeholder values like -99.99, which indicate missing data).

Below are the functions used in the project and a brief description of what they do:

□ `plot_catchment(df, catchment)` - This function takes in a DataFrame containing hydrological data and a specific catchment identifier. It generates two plots for the specified catchment: Streamflow vs. Rainfall (visualising how rainfall correlates with streamflow over time) and Streamflow vs. Maximum Temperature (showing the relationship between streamflow and daily maximum temperature)

□ `find_95(streamflow_vals)` - This function computes the 95th percentile of a given list or array of numerical values.

□ `valid(prompt)` - This function checks whether the input provided to the flood prediction model is valid. It ensures that

the necessary features are present, correctly formatted, and free of invalid or missing values.

□ `sigmoid(z)` - returns $1/(1+e^{\text{input number}})$. This is required to convert values into numbers between 0 and 1

□ `logistic_regression_train (X, y, learning_rate=0.01, iterations=1000)` - This function trains a logistic regression model using the provided feature matrix X and target labels y. It applies gradient descent to iteratively update the model's weights and bias to minimize the loss function.

□ `logistic_regression_predict (X, weights, bias, threshold=0.5)` - This function uses the trained logistic regression model to make predictions. It computes the probability of a positive class (e.g., flood occurrence) for each input sample using the sigmoid function applied to the linear combination of features, weights, and bias. If the resulting probability exceeds the specified threshold (default is 0.5), the function returns 1 (indicating a flood); otherwise, it returns 0 (no flood).

The main logic loop of the code is as follows:

A. Data Ingestion and Processing

- `extract_top5.py`: Filters and saves only the top five catchments for efficient processing
- `main.py`: Central driver script that loads streamflow, rainfall, and temperature datasets and transforms them into long-form data using `pandas.melt()`

B. Feature Engineering and Labelling

- Streamflow threshold computed per catchment via a `find_95()` function
- New features created: `rain_lag1`, `rain_lag2`, `temp_lag1`, `rain_3day_sum`
- NaN filtering and sorting ensure temporal consistency

C. Model Implementation

- Custom logistic regression (`flood_model.py`) coded using NumPy, applying sigmoid activation
- Training logic via `logistic_regression_train()` and prediction via `logistic_regression_predict()`
- Model performance metrics computed using `sklearn.metrics`: accuracy, precision, recall, F1-score, and confusion matrix

D. Visualisation

- `plot.py`: Contains `plot_catchment()` function, which generates dual-axis plots for streamflow vs rainfall and streamflow vs temperature

- Plots include log-scaled streamflow, rainfall bars, and overlaid flood threshold lines, saved per catchment

E. Interactive Prediction

- `predict_flood.py`: Allows user to input real-time values and returns flood risk via command line
- Writes logs to `log.csv`, including timestamp, inputs, and prediction

This modular design enables reproducibility and extendibility. New models or data sources can be integrated by swapping out the logistic regression backend or linking to live weather APIs.

IV. RESULTS VISUALISATION AND DISCUSSION

In a nutshell the model is run on 80% of the data and 20% of the data was used to test how well our model worked. Our model achieved an overall accuracy of 95.4%, which may initially appear as strong, however, taking a closer look at the precision reveals important limitations:

- Precision is 58.4% indicates that when the model predicts a flood, it is only correct around 58% of the time.
- The recall is 29.2% meaning the model only identifies 29.2% of the actual flood days — missing over 70% of them.

Precision and recall are more informative here as indicated in *Figure 1* showing 274 false positives and 935 false negatives, revealing a tendency to under-predict rare events and have an impact on the accuracy of the flood prediction results.

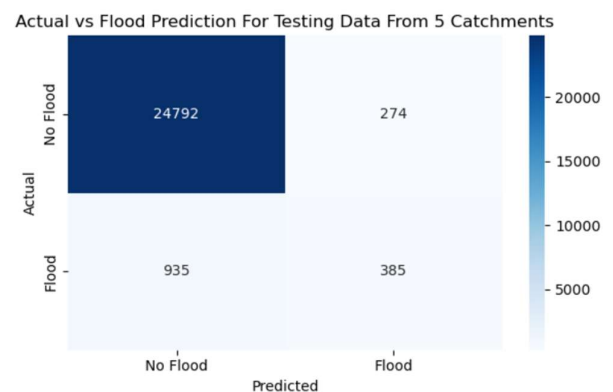


Figure 1

The low recall is particularly concerning in the context of flood prediction, as failing to detect these flood events can have serious consequences on the environment. Improving the recall should be a priority to ensure that the model can reliably alert the surrounding communities to potential flooding through signs and warnings.

While our model is focused solely on previous rainfall and environmental temperature as the indicators of flooding, it is important to consider and acknowledge that various other

factors can influence flood behaviour in rivers and catchment areas.

For example, the Base Flow Index (BFI) reflects the proportion of baseflow in streamflow and is influenced by external factors such as water resource management, watershed characteristics, and surrounding land usage. A low BFI typically indicates a flashy catchment; this means the river responds more rapidly to rainfall events and reaches peak flow at a faster rate[2]. Similarly, a high FLASH ID is also indicative of flashiness in catchment response in relation to the flood level.

Another key variable to flood prediction is response time (Resptime), which represents how quickly a catchment will respond to the impacts of rainfall. This is mainly governed by catchment geomorphology, land cover, and soil properties [3].

Additionally, Total Runoff Ratio (TotalRR) measures how efficiently a catchment will impact the rainfall and force it into water runoff in the environment. A higher TotalRR indicates more efficient runoff and generally indicates higher flood risks. This efficiency is shaped by the permeability of the surrounding soil, vegetation cover, and the land slope of the area[4].

In short, many hydrological and physical characteristics can significantly impact flood likelihood, however we did not include them into our model as we tried to take in a perfect world scenario without the practical constraints, such as data availability and scope. This omission helps explain our model's lower precision and recall, despite its high overall accuracy. Expanding our model to incorporate these additional variables could significantly improve its ability to detect flood events making it a more reliable and accurate tool to use in this manner.

The above-mentioned trend is consistent with flashier catchments such as 915206A and 915011A, both of which have:

- **Low Base Flow Index (BFI):** 0.04, 0.10 → indicates rapid runoff response
- **High Flash Index:** 0.98, 0.72 → indicates sudden streamflow peaks
- **Short Response Time:** 1 day → minimal lag between rainfall and flooding

For external validation, we tested the model on an entirely unseen catchment: **224213 (Dargo River, VIC)**. The results were:

- Accuracy: **87.2%**
- Precision: **20.2%**
- Recall: **57.3%**

Figure 2 confirms the performance shift showcasing that more flood days were caught, but at the cost of increased false alarms. More importantly, Figure 3 (timeline plot) illustrates the flood detection alignment over time, drawing attention to the spikes in prediction that often coincide with flood periods that were previously provided.

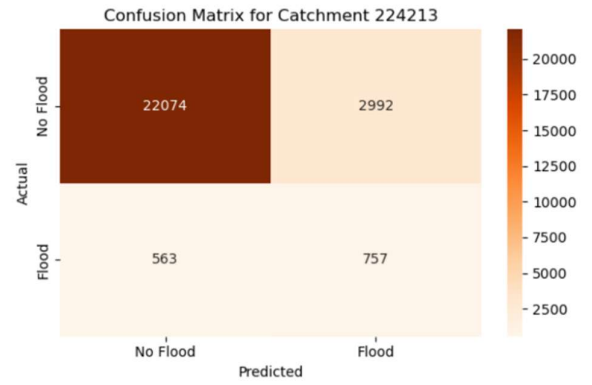


Figure 2

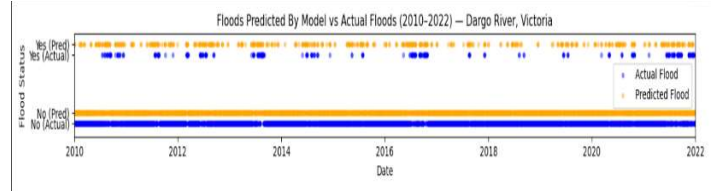


Figure 3

Another significant plot is the **912101A subplot (Figure 4.1 and 4.2)** showing streamflow, rainfall, and temperature. The use of log-scaled streamflow effectively highlights flood outliers, with the overlaid threshold and the rainfall bars clearly corresponding the storm events to flood conditions in the given area. This visual validates our choice of lagged features and justifies the 95th percentile threshold that we've identified.

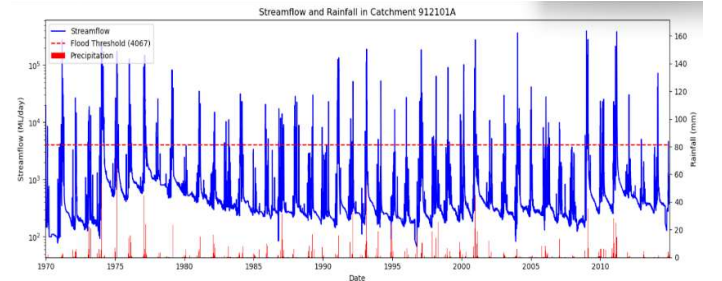


Figure 4.1

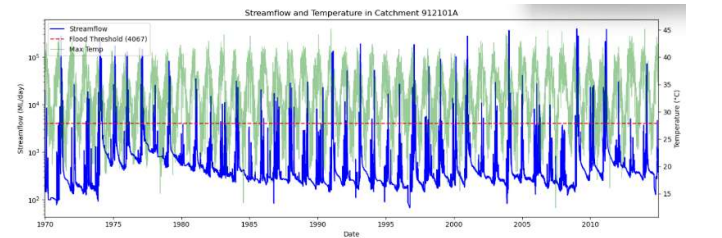


Figure 4.2

V. LIMITATIONS AND FUTURE DIRECTIONS

- Our model assumes uniform catchment behaviour across time (no seasonal calibration)
- Excluding soil moisture, topography, and land use

- No spatial correlation (i.e., the upstream/downstream flow not considered)
- Threshold-based flood definition does not clearly distinguish between the nuisance and extreme flooding

Future improvements could integrate:

- Random Forests or XGBoost to capture nonlinearities
- Seasonal adjustment given via sliding percentile windows
- Inclusion of soil saturation/NDVI via remote sensing
- Real-time weather feeds from BoM/NOAA

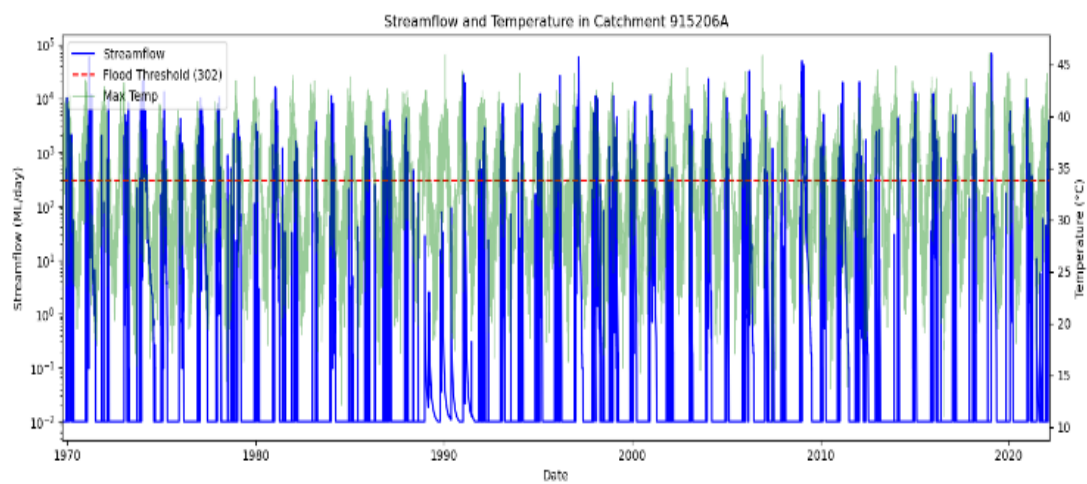
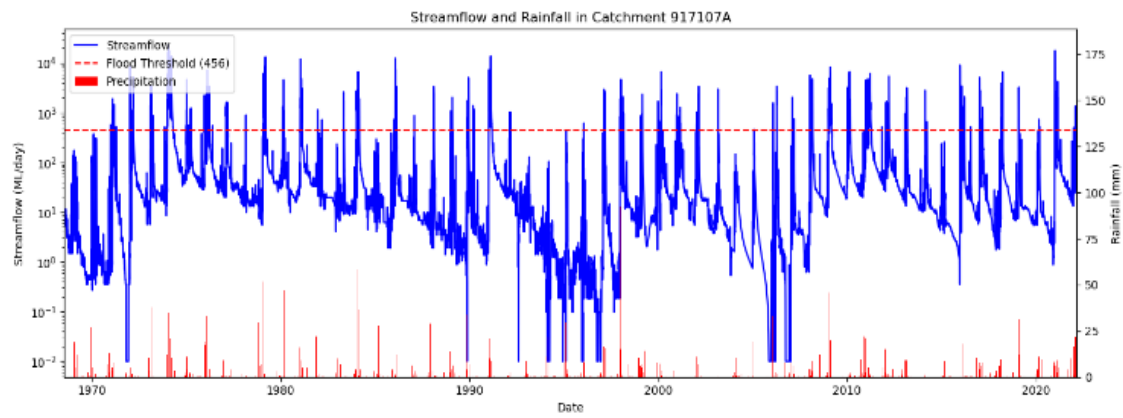
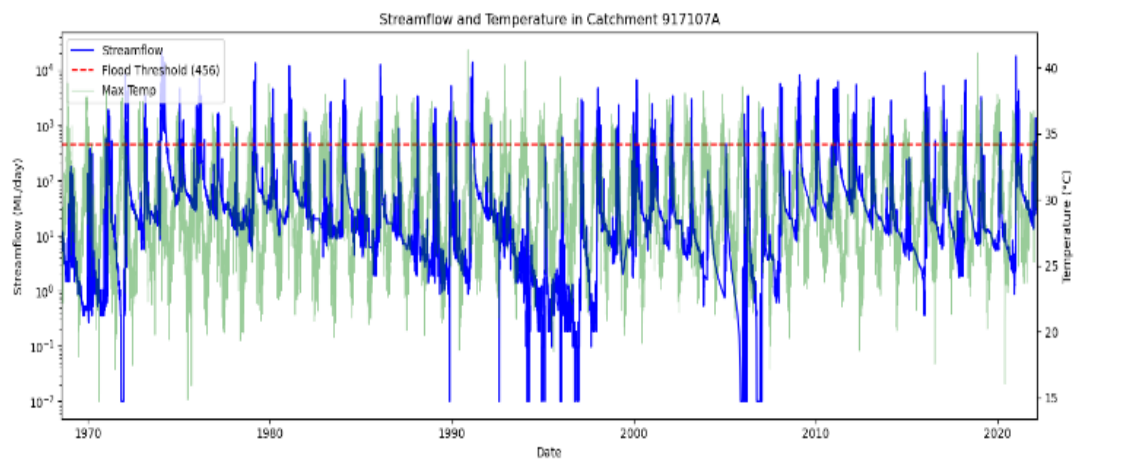
This study provides a replicable foundation for operational-usage lightweight and data-driven flood risk tools to help in the prediction of floods. With minor refinements, the methodology can support early-warning infrastructure and

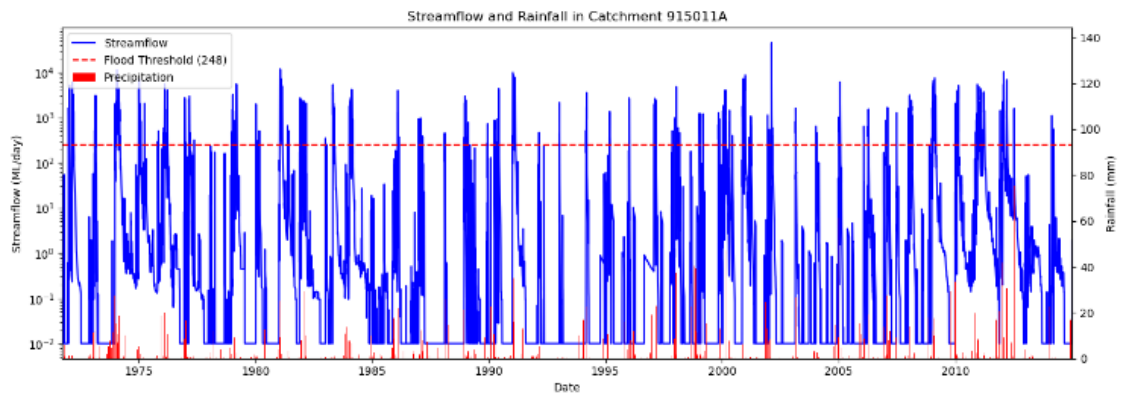
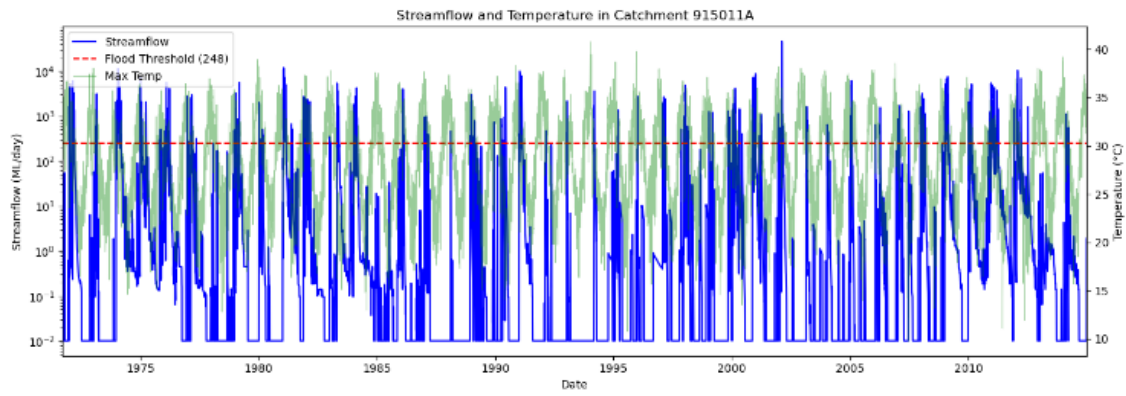
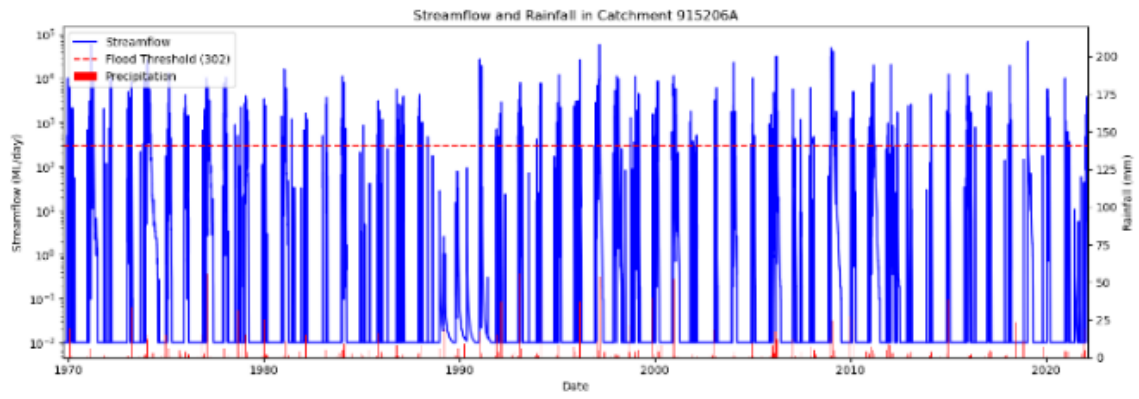
serve as a prediction and teaching tool in environmental sustainability programs to help with the impacts of flooding

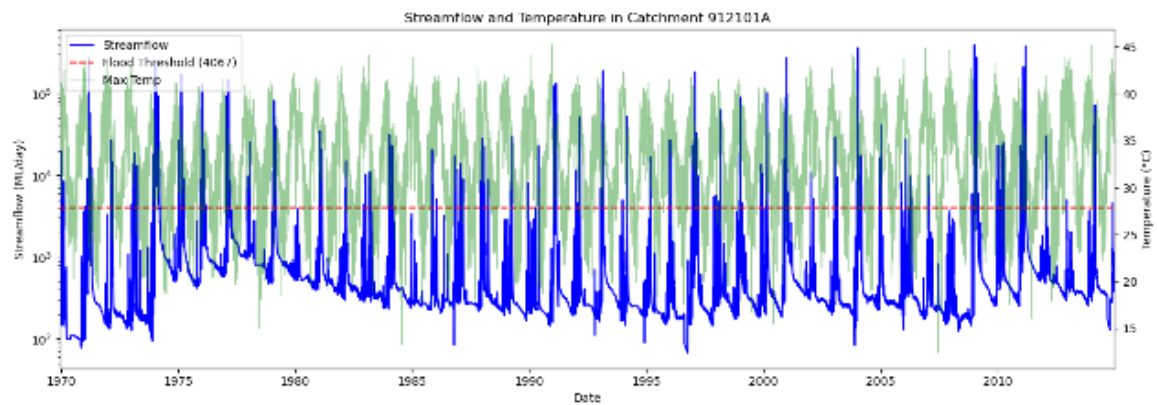
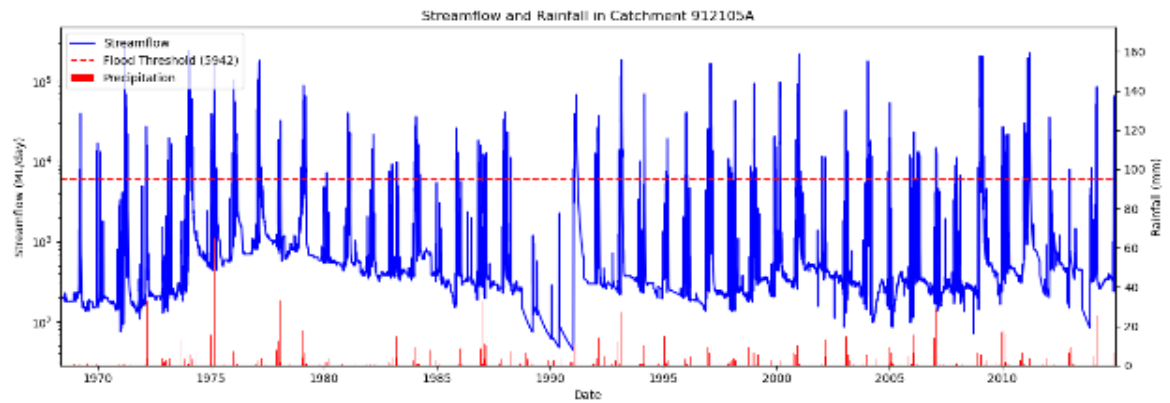
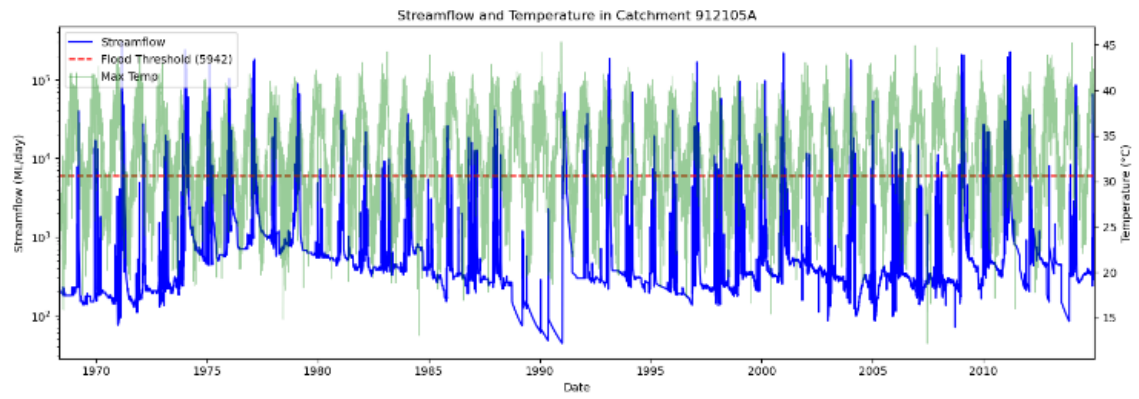
REFERENCES

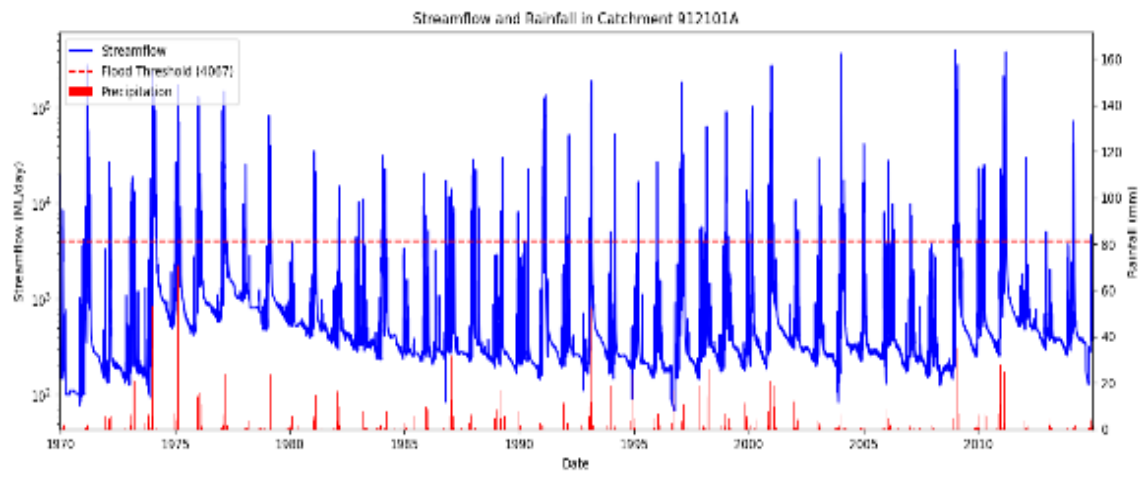
- [1] Addor, N. et al. (2021). Robust changes in flow regimes in the CAMELS-AUS dataset. *Hydrology and Earth System Sciences*, 25(9), 5355–5380. <https://hess.copernicus.org/articles/25/5355/2021/>
 - [2] Royappen, M. et al. (2013). Catchment response time: An assessment of geomorphological controls. *Hydrological Sciences Journal*, 58(6), 1261–1274. <https://www.tandfonline.com/doi/full/10.1080/02626667.2013.866712>
 - [3] Zhou, F. et al. (2021). Investigating runoff generation processes using rainfall-runoff modeling in diverse Australian catchments. *Water Cycle*, 2, 1–13. <https://www.sciencedirect.com/science/article/pii/S2095633921000885>
 - [4] Bates, P. et al. (2021). Advances in flood forecasting: New tools and frameworks. *Environmental Modelling & Software*, 145, 105211. <https://doi.org/10.1016/j.envsoft.2021.105211>
- Peel, M. C. et al. (2001). The impact of climate variability and catchment conditions on streamflow predictions in southeastern Australia. *Hydrological Processes*, 15(10), 1903–1918. <https://doi.org/10.1002/hyp.596>

Appendix:










```

# Plot.py: python file which holds out plot functions!
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import os
import glob

def find_95(streamflow_vals):
    return np.percentile(streamflow_vals, 95)

# Plotting function
def plot_catchment(df, catchment):

    df_catch = df[df['catchment_id'] == catchment]
    df_sf = df_catch[df_catch['streamflow'] > -99.99].copy()
    df_sf['streamflow'] += 0.01 # Avoid log(0) errors by adding a small amount (0.01)

    threshold = find_95(df_sf['streamflow'].values)
    df_sf['flood'] = df_sf['streamflow'] >= threshold # streamflow only dataframe
    df_rain = df_catch[df_catch['precipitation'] > -99.99].copy() # rainfall only dataframe
    df_temp = df_catch[df_catch['temperature'] > -99.99].copy() # temperature only dataframe

    # plot streamflow with left side y axis
    fig, ax1 = plt.subplots(figsize=(14, 5))
    ax1.plot(df_sf['date'], df_sf['streamflow'], color='blue', label='Streamflow')
    ax1.axhline(y=threshold, color='red', linestyle='--', label=f'Flood Threshold ({threshold:.0f})')
    ax1.set_yscale('log')
    ax1.set_ylabel('Streamflow (ML/day)')
    ax1.set_xlabel('Date')
    ax1.set_xlim(df_sf['date'].min(), df_sf['date'].max())

    # plot the rainfall with the right side y axis
    ax2 = ax1.twinx()
    ax2.bar(df_rain['date'], df_rain['precipitation'], color='red', label='Precipitation')
    ax2.set_ylabel('Rainfall (mm)')
    maxrain = df_rain['precipitation'].max()
    ax2.set_ylim(0, maxrain)

    # add legend
    lines, labels = ax1.get_legend_handles_labels()
    lines2, labels2 = ax2.get_legend_handles_labels()
    ax1.legend(lines + lines2, labels + labels2, loc='upper left')

    # plot and save figure
    plt.title(f'Streamflow and Rainfall in Catchment {catchment}')
    plt.tight_layout()
    plt.savefig(f'plots/{catchment}_sf_rainfall_combined.png')
    plt.close()

    # do the same thing but now plot streamflow with max temperatures
    fig, ax1 = plt.subplots(figsize=(14, 5))
    ax1.plot(df_sf['date'], df_sf['streamflow'], color='blue', label='Streamflow')
    ax1.axhline(y=threshold, color='red', linestyle='--', label=f'Flood Threshold ({threshold:.0f})')
    ax1.set_yscale('log')
    ax1.set_ylabel('Streamflow (ML/day)')
    ax1.set_xlabel('Date')
    ax1.set_xlim(df_sf['date'].min(), df_sf['date'].max())

    ax2 = ax1.twinx()
    ax2.plot(df_temp['date'], df_temp['temperature'], color='green', alpha=0.4, linewidth=0.8, label='Max Temp')
    ax2.set_ylabel('Temperature (C)')

    lines, labels = ax1.get_legend_handles_labels()
    lines2, labels2 = ax2.get_legend_handles_labels()
    ax1.legend(lines + lines2, labels + labels2, loc='upper left')

    plt.title(f'Streamflow and Temperature in Catchment {catchment}')
    plt.tight_layout()
    plt.savefig(f'plots/{catchment}_sf_temp_combined.png')
    plt.close()

```

```

# predict flood file: main deliverable file which the end user can run to predict a flood!

# ===== HOW TO USE =====
# Run this script in the terminal: python predict_flood.py
# Enter values when prompted

import joblib
import numpy as np
import csv
from datetime import datetime
import os

def valid(prompt):
    while True:
        try:
            value = float(input(prompt))
            if value < 0: raise ValueError
            return value
        except:
            print("Please enter a valid non-negative number.")

model = joblib.load("flood_model.pk1")
weights = model['weights'] #imports our b1, b2 etc
bias = model['bias'] # imports b0

print("***** WELCOME TO THE FLOOD PREDICTOR! *****")

# inputs
rain_today = valid("Today's rainfall (mm): ")
rain_lag1 = valid("Yesterday's rainfall (mm): ")
rain_lag2 = valid("Rainfall 2 days ago (mm): ")
temp_lag1 = valid("Yesterday's temperature (C): ")

rain_3day_sum = rain_today + rain_lag1 + rain_lag2

features = np.array([[rain_lag1, rain_lag2, temp_lag1, rain_3day_sum]])
linear_model = np.dot(features, weights) + bias
p = 1/(1+np.exp(-linear_model)) #calculate p using logistic regression
prediction = int(p >= 0.5)

print(f'\nFlood Prediction for Tomorrow: {'Yes' if prediction else 'No'})
print(f'Probability of a flood tomorrow: {p[0]:.2f}')
write_header = not os.path.exists('log.csv')
with open("log.csv", "a", newline='') as file:
    writer = csv.writer(file)

    if write_header:
        writer.writerow([
            "Time", "rain_today_mm", "rain_lag1_mm", "rain_lag2_mm", "temp_lag1_C", "rain_3day_sum_mm", "Flood Probability", "Prediction"])

    writer.writerow([datetime.now().isoformat(), rain_today, rain_lag1, rain_lag2, temp_lag1, rain_3day_sum, float(p[0]), prediction])

```

```

import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import glob
import os
import joblib
import seaborn as sns

from sklearn.metrics import accuracy_score, precision_score, recall_score
from plot import plot_catchment
from flood_model import sigmoid, logistic_regression_train, logistic_regression_predict
from sklearn.metrics import confusion_matrix

# loading data into a dataframe
sf = pd.read_csv('data/streamflow_TOP5.csv', parse_dates=['date']) # sf = streamflow!!
rain = pd.read_csv('data/precipitation_TOP5.csv', parse_dates=['date'])
temp = pd.read_csv('data/temperature_TOP5.csv', parse_dates=['date'])

# Melt each to long format
sf_l = sf.melt(id_vars='date', var_name='catchment_id', value_name='streamflow')
rain_l = rain.melt(id_vars='date', var_name='catchment_id', value_name='precipitation')
temp_l = temp.melt(id_vars='date', var_name='catchment_id', value_name='temperature')

# create a mega-dataframe to hold our data
df = pd.merge(sf_l, rain_l, on=['date', 'catchment_id'])
df = pd.merge(df, temp_l, on=['date', 'catchment_id'])

# adding lagged features

# Sort data to ensure correct lagging
df = df.sort_values(by=['catchment_id', 'date'])

# Create lagged features within each catchment group
df['rain_lag1'] = df.groupby('catchment_id')['precipitation'].shift(1) # rain on previous day
df['rain_lag2'] = df.groupby('catchment_id')['precipitation'].shift(2) # rain 2 days before
df['temp_lag1'] = df.groupby('catchment_id')['temperature'].shift(1) # temp on previous day

# rain 3 day sum:
df['rain_3day_sum'] = (
    df.groupby('catchment_id')['precipitation']
    .shift(1)
    .rolling(window=3, min_periods=1)
    .sum()
    .reset_index(level=0, drop=True)
)

# Definition of 'flood day': where streamflow on that day > 95th percentile of all streamflow in that catchment
def find_95(streamflow_vals):
    return np.percentile(streamflow_vals, 95)

thresholds = {
    "912101A": find_95(df[df["catchment_id"] == "912101A"]['streamflow'].tolist()),
    "912105A": find_95(df[df["catchment_id"] == "912105A"]['streamflow'].tolist()),
    "915011A": find_95(df[df["catchment_id"] == "915011A"]['streamflow'].tolist()),
    "915206A": find_95(df[df["catchment_id"] == "915206A"]['streamflow'].tolist()),
    "917107A": find_95(df[df["catchment_id"] == "917107A"]['streamflow'].tolist())
}

df['threshold'] = df['catchment_id'].map(thresholds) # store the threshold value for each catchment in the dataframe
df['flood'] = df['streamflow'] > df['threshold'] # set flood value in dataframe to be True if streamflow on that day exceeds flood threshold

# Plotting: we plot rainfall and temperature on the same plot as streamflow to show how these variables correlate with flooding
for c in df['catchment_id'].unique():
    plot_catchment(df, c)

# run the model:

df_c = df.dropna(subset=['rain_lag1', 'rain_lag2', 'temp_lag1', 'rain_3day_sum', 'flood']).copy()
# Define input and target
features = df_c[['rain_lag1', 'rain_lag2', 'temp_lag1', 'rain_3day_sum']].values
target = df_c['flood'].astype(int).values

# Split: 80% train, 20% test (no shuffle, keep temporal order)
split_idx = int(0.8 * len(features))
X_train, X_test = features[:split_idx], features[split_idx:]
y_train, y_test = target[:split_idx], target[split_idx:]

# Train model
weights, bias = logistic_regression_train(X_train, y_train)

# Predict
y_pred = logistic_regression_predict(X_test, weights, bias) #y_pred is

# Evaluate accuracy, precision and recall values, and F-1 recall_score
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

f1_score = (2* precision * recall)/(precision + recall)

# plot predicted vs actual flood days for all 5 catchments' testing data on the same confusion matrix
# add plot code here from scratch in this file main.py

cm = confusion_matrix(y_test, y_pred)

```

```

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No Flood', 'Flood'],
            yticklabels=['No Flood', 'Flood'])

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Actual vs Flood Prediction For Testing Data From 5 Catchments')
plt.tight_layout()
plt.show()
plt.savefig('Predicted vs Actual Flood Days.png') # save confusion matrix

# true positive, true negative, false positives, false negatives
TP = np.sum((y_test == 1) & (y_pred == 1))
TN = np.sum((y_test == 0) & (y_pred == 0))
FP = np.sum((y_test == 0) & (y_pred == 1))
FN = np.sum((y_test == 1) & (y_pred == 0))
print(f"Confusion Matrix:\nTP: {TP}, FP: {FP}, TN: {TN}, FN: {FN}")

# saving the model
joblib.dump({'weights': weights, 'bias': bias}, 'flood_model.pk1')
print('Model updated!')

#display weight+bias+variables
print(f"Bias (b0): {bias:.4f}")
for i, w in enumerate(weights, start=1):
    print(f"Weight b{i}: {w:.4f}")

# print of evalutaion
print('')
print(f'Accuracy Value: {accuracy:.3f}')
print(f'Precision Value: {precision:.3f}')
print(f'Recall Value: {recall:.3f}')
print(f'F1 Score : {f1_score:.3f}')

# generate predicted vs actual flood for random catchment using our model

df_224 = pd.read_csv('data/224213_data.csv', parse_dates=['date'])
df_224 = df_224.sort_values(by='date')

df_224['actual_flood'] = df_224['streamflow'] > np.percentile(df_224['streamflow'], 95)
df_224['rain_lag1'] = df_224['rainfall'].shift(1)
df_224['rain_lag2'] = df_224['rainfall'].shift(2)
df_224['rain_3day_sum'] = df_224['rainfall'].shift(1).rolling(window=3, min_periods=1).sum()
df_224['temp_lag1'] = df_224['max_temp'].shift(1)

df_224_clean = df_224.dropna(subset=['rain_lag1', 'rain_lag2', 'rain_3day_sum', 'temp_lag1'])

X = df_224_clean[['rain_lag1', 'rain_lag2', 'temp_lag1', 'rain_3day_sum']].values

df_224_clean['predicted_flood'] = logistic_regression_predict(X, weights, bias)

# plotting
cm = confusion_matrix(df_224_clean['actual_flood'], df_224_clean['predicted_flood'])

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Oranges',
            xticklabels=['No Flood', 'Flood'],
            yticklabels=['No Flood', 'Flood'])

plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix for Catchment 224213')
plt.tight_layout()
plt.savefig('test_confusion_matrix.png')
plt.show()

print('----- EVALUATION of MODEL PERFORMANCE on NEW CATCHMENT - DARGO RIVER, VICTORIA -----')
print('')
print(f'Accuracy: {accuracy_score(df_224_clean['actual_flood'], df_224_clean['predicted_flood'])*100:.3f}%')
print(f'Precision: % of predicted floods that were actual floods: {precision_score(df_224_clean['actual_flood'], df_224_clean['predicted_flood'])*100:.3f}%')
print(f'Recall: % of actual floods the model correctly found: {recall_score(df_224_clean['actual_flood'], df_224_clean['predicted_flood'])*100:.3f}%')

# plot as a timeline for a small range of recent dates
df_zoom = df_224_clean[df_224_clean['date'].between('2010-01-01', '2022-01-01')]
plt.figure(figsize=(14, 2))

actual_y = df_zoom['actual_flood']
predicted_y = df_zoom['predicted_flood'] + 0.15 # add an offset val

plt.scatter(df_zoom['date'], actual_y, label='Actual Flood', s=10, color='blue', alpha=0.4)
plt.scatter(df_zoom['date'], predicted_y, label='Predicted Flood', s=10, color='orange', alpha=0.4)

plt.yticks([0, 0.15, 1, 1.15], ['No (Actual)', 'No (Pred)', 'Yes (Actual)', 'Yes (Pred)'])
plt.ylim(-0.2, 1.3)
plt.xlim(df_zoom['date'].min(), df_zoom['date'].max())

plt.title("Floods Predicted By Model vs Actual Floods (2010-2022) – Dargo River, Victoria")
plt.xlabel("Date")
plt.ylabel("Flood Status")
plt.legend()
plt.tight_layout()
plt.savefig("224213_timeseries.png")
plt.show()

```

```
import numpy as np
import pandas as pd
import matplotlib as plt

# logistic regression model code

# sigmoid function
def sigmoid(z):
    return 1/(1 +np.exp(-z))

def logistic_regression_train(X, y, learning_rate=0.01, iterations=1000):
    m, n = X.shape #n is num of features (eg. rain_lag1, rain_lag2 etc.)
    weights = np.zeros(n) #all weights initialise to 0, weights = b1 ....
    bias = 0 #final intercept variable

    for i in range(iterations):
        linear_model = np.dot(X, weights) + bias
        predictions = sigmoid(linear_model) #calculates predicted value for each row using sigmoid(X x w + b)

        err = predictions-y

        dw = (1 / m) * np.dot(X.T, err) #calculates how much to change weight dw and bias db based on avg error
        db = (1 / m) * np.sum(err)

        weights -= learning_rate * dw #update value of weights, bias
        bias -= learning_rate * db #b0

    return weights, bias

def logistic_regression_predict(X, weights, bias, threshold=0.5):
    probs = sigmoid(np.dot(X, weights) + bias) # calculates how likely each row is a flood
    return (probs >= threshold).astype(int) # if prob > 0.5, predicts flood (outputs '1')
```

```

# This is code I used to extract data from 5 catchments from each of the relevant files in the CAMELS AUS dataset
# Note: the filepaths are specific to my computer
# This produced the files under the data folder in this Ed workspace

import pandas as pd
import os

#top 5 catchments (match column names exactly)
top_ids = ['912101A', '912105A', '915011A', '915206A', '917107A']

# streamflow
sf = pd.read_csv(r"C:\Users\hawts\OneDrive\Documents\Personal\USYD\Sem 1 2025\ENGG1810\Assessment\Grand_challenge\data\03_streamflow\03_streamflow\streamf]
sf['date'] = pd.to_datetime(sf[['year', 'month', 'day']])
sf_top5 = sf[['date'] + top_ids]

sf_top5.to_csv('data_small/streamflow_TOP5.csv', index=False)

# ppt
precip = pd.read_csv(r"C:\Users\hawts\OneDrive\Documents\Personal\USYD\Sem 1 2025\ENGG1810\Assessment\Grand_challenge\data\05_hydrometeorology\05_hydromete]
precip['date'] = pd.to_datetime(precip[['year', 'month', 'day']])
precip_top5 = precip[['date'] + top_ids]
precip_top5.to_csv('data_small/precipitation_TOP5.csv', index=False)

# temp
temp = pd.read_csv(r"C:\Users\hawts\OneDrive\Documents\Personal\USYD\Sem 1 2025\ENGG1810\Assessment\Grand_challenge\data\05_hydrometeorology\05_hydrometeor]
temp['date'] = pd.to_datetime(temp[['year', 'month', 'day']])
temp_top5 = temp[['date'] + top_ids]
temp_top5.to_csv('data_small/temperature_TOP5.csv', index=False)

# metadata for each catchment!
meta = pd.read_csv(r"C:\Users\hawts\OneDrive\Documents\Personal\USYD\Sem 1 2025\ENGG1810\Assessment\Grand_challenge\data\01_id_name_metadata\01_id_name_me]
meta['station_id'] = meta['station_id'].astype(str).str.strip()
top_ids_cleaned = [cid.strip() for cid in top_ids]
meta_top5 = meta[meta['station_id'].isin(top_ids_cleaned)]
meta_top5.to_csv('data_small/metadata_TOP5.csv', index=False)

# sigs
signatures_path = (r"C:\Users\hawts\OneDrive\Documents\Personal\USYD\Sem 1 2025\ENGG1810\Assessment\Grand_challenge\data\03_streamflow\03_streamflow\stream]
signatures = pd.read_csv(signatures_path)
signatures['CatchID'] = signatures['CatchID'].astype(str).str.strip()
sig_top5 = signatures[signatures['CatchID'].isin(top_ids)]
sig_top5.to_csv('data_small/streamflow_signatures_TOP5.csv', index=False)

# Additional catchment: load a new, random test catchment to plot the accuracy of our prediction model
# catchment 224213: Dargo River, South east coast (Victoria), part of the Mitchell-Thompson Rivers
sf_224213 = sf[['date', '224213']].rename(columns={'224213': 'streamflow'})
rain_224213 = precip[['date', '224213']].rename(columns={'224213': 'rainfall'})
temp_224213 = temp[['date', '224213']].rename(columns={'224213': 'max_temp'})
df_merged = sf_224213.merge(rain_224213, on='date').merge(temp_224213, on='date')
df_merged.to_csv('data_small/224213_data.csv', index=False)

print('done') # simple flag

```